🎯 "I've Never Done a CV Project - Where Do I Start?"

Don't worry! Follow this step-by-step guide and you'll be fine.

🗓️ WEEK-BY-WEEK ROADMAP

Week 9 (Oct 16-23): Choose & Plan

Day 1-2: Pick Your Project

- Browse Tier 1 project ideas
- Choose one that interests you
- Find a teammate (optional but recommended!)
- Create a team chat/Discord

How to choose:

- What sounds fun?
- What would you use in real life?
- What has available datasets?

Day 3-4: Find Your Dataset

- Google: "[your project] dataset"
- Check these sites:
    - Kaggle Datasets
    - Roboflow Universe
    - Hugging Face Datasets
    - Google Dataset Search
- Download a small sample (100-500 images)
- Look at the images - do they make sense?

Red flags to avoid:

- Dataset requires payment
- Dataset is too large (>100GB)
- Images are low quality or mislabeled
- Can't find any dataset at all

Day 5-6: Set Up Your Environment

Option 1: Google Colab (Recommended for beginners)

Open new notebook at colab.research.google.com, click "New Notebook"

Test GPU access:

```
!nvidia-smi
```

Install basic packages:

```
!pip install ultralytics opencv-python matplotlib
```

Option 2: Local Jupyter

Install Anaconda first, then create environment:

```
conda create -n cv_project python=3.10
conda activate cv_project
pip install jupyter ultralytics opencv-python matplotlib torch torchvision
```

Day 7: Create GitHub Repository

Step-by-step:

1. Go to github.com
2. Click "New Repository"
3. Name it: `cv-project-[your-topic]`
4. Check "Add README"
5. Click "Create Repository"
6. Add basic structure:

```
cv-project-inventory/
├── README.md
├── requirements.txt
├── notebooks/
├── data/
└── docs/
```

7. Fill in README using the template (see Midterm guide)

Week 10 (Oct 30): Proposal Presentation

Day 1-2: Make Slides

- Use PowerPoint/Google Slides
- Follow the 8-10 slide template
- Keep text minimal, use images
- Include your system diagram

Pro tip: Steal slide templates from Google Slides!

Day 3: Practice Presentation

- Time yourself - aim for 4-5 minutes
- Practice with your teammate
- Record yourself (phone is fine)
- Watch it back - fix awkward parts

Day 4: Submit & Present

- Upload slides to GitHub docs/ folder
- Submit GitHub link on Canvas
- Present in class!
- Breathe! You've got this! 😊

Week 11 (Nov 6-13): Get Model Working

Day 1-2: Load Pre-trained Model

For Object Detection (YOLO):

In Google Colab or Jupyter:

```
# Install
!pip install ultralytics

# Load model
from ultralytics import YOLO
model = YOLO('yolov8n.pt')  # 'n' = nano (smallest/fastest)

# Test on sample image
results = model('https://ultralytics.com/images/bus.jpg')
results[0].show()  # Should show detected objects!

print("✅ Model is working!")
```

For Classification (ResNet):

```
import torch
import torchvision.models as models
from PIL import Image
import torchvision.transforms as transforms

# Load pre-trained model
model = models.resnet50(pretrained=True)
model.eval()

# Load and preprocess image
img = Image.open('test.jpg')
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
input_tensor = preprocess(img)
input_batch = input_tensor.unsqueeze(0)

# Predict
with torch.no_grad():
    output = model(input_batch)

print("✅ Model is working!")
```

Day 3-5: Organize Your Dataset

Standard structure:

```
data/
├── train/
│   ├── images/
│   │   ├── img001.jpg
│   │   ├── img002.jpg
│   └── labels/
│       ├── img001.txt
│       ├── img002.txt
├── val/
│   ├── images/
```

```
│   └── labels/
└── test/
    ├── images/
    └── labels/
```

For YOLO, create data.yaml:

```
# data.yaml
path: ./data
train: train/images
val: val/images
test: test/images

# Number of classes
nc: 3

# Class names
names: ['class1', 'class2', 'class3']
```

Day 6-7: Fine-tune Model (If Needed)

For YOLO:

```
from ultralytics import YOLO

# Load pre-trained model
model = YOLO('yolov8n.pt')

# Train on your data
results = model.train(
    data='data.yaml',
    epochs=50,           # Number of training epochs
    imgsz=640,          # Image size
    batch=16,           # Batch size (reduce if out of memory)
    name='my_model'     # Name for this training run
)

# Results will be in: runs/detect/my_model/
```

Common issues:

- "CUDA out of memory" → Reduce batch size to 8 or 4
- "No labels found" → Check your data.yaml paths

- Training is slow → Use smaller model (yolov8n) or fewer epochs

---

Week 12 (Nov 13-20): Test & Improve

Day 1-3: Evaluate Performance

For YOLO:

```
# Load your trained model
model = YOLO('runs/detect/my_model/weights/best.pt')

# Validate on test set
metrics = model.val()

# Print results
print(f"mAP50: {metrics.box.map50:.3f}")
print(f"mAP50-95: {metrics.box.map:.3f}")
print(f"Precision: {metrics.box.mp:.3f}")
print(f"Recall: {metrics.box.mr:.3f}")
```

For Classification:

```
from sklearn.metrics import accuracy_score, classification_report

# Make predictions on test set
predictions = []
true_labels = []

for image, label in test_dataset:
    pred = model.predict(image)
    predictions.append(pred)
    true_labels.append(label)

# Calculate metrics
accuracy = accuracy_score(true_labels, predictions)
print(f"Accuracy: {accuracy:.3f}")
print(classification_report(true_labels, predictions))
```

Day 4-7: Improve Performance

If accuracy is low, try:

1. Data Augmentation:

```
# For YOLO, add to training:
model.train(
    data='data.yaml',
    epochs=50,
    augment=True,  # Enable augmentation
    hsv_h=0.015,   # Hue augmentation
    hsv_s=0.7,     # Saturation
    hsv_v=0.4,     # Value
    degrees=0.0,   # Rotation
    translate=0.1, # Translation
    scale=0.5,     # Scaling
    flipud=0.5,    # Flip up-down
    fliplr=0.5,    # Flip left-right
)
```

2. More Training Data:

- Collect more images
- Use data from similar datasets
- Create synthetic data

3. Try Different Model Size:

```
# Instead of yolov8n.pt (nano), try:
model = YOLO('yolov8s.pt')  # Small
model = YOLO('yolov8m.pt')  # Medium (slower but more accurate)
```

4. Tune Hyperparameters:

```
model.train(
    data='data.yaml',
    epochs=100,      # Try more epochs
    lr0=0.001,       # Learning rate
    patience=20,     # Early stopping patience
)
```

Week 13 (Nov 20-27): Create Demo

Day 1-2: Make Demo Notebook

Create: `notebooks/04_demo.ipynb`

Cell 1: Setup

```
from ultralytics import YOLO
from PIL import Image
import matplotlib.pyplot as plt
```

Cell 2: Load Model

```
model = YOLO('runs/detect/my_model/weights/best.pt')
print("✅ Model loaded!")
```

Cell 3: Demo Function

```
def run_demo(image_path):
    """Run detection on an image and display results"""
    # Run inference
    results = model(image_path)

    # Display results
    results[0].show()

    # Print detections
    for box in results[0].boxes:
        class_id = int(box.cls[0])
        confidence = float(box.conf[0])
        class_name = model.names[class_id]
        print(f"Detected: {class_name} ({confidence:.2f})")
```

Cell 4: Test on Sample Images

```
run_demo('data/sample/test1.jpg')
run_demo('data/sample/test2.jpg')
run_demo('data/sample/test3.jpg')
```

Cell 5: Statistics

```
total_images = len(test_images)
total_detections = count_all_detections()
print(f"Processed {total_images} images")
print(f"Made {total_detections} detections")
```

Day 3-4: Record Demo Video

Setup:

1. Open Zoom (or OBS, QuickTime)
2. Click "New Meeting"
3. Click "Share Screen" → Share your full screen
4. Click "Record"

Script:

[30 seconds] Introduction

```
"Hi, I'm [Name]. This is [Project Name].
It solves [problem] by [solution]."
```

[2-3 minutes] Live Demo

```
"Let me show you how it works..."
[Run your notebook, show 3-5 test cases]
[Explain what's happening]
[Show both successes AND failures]
```

[1 minute] Results

```
"Here are our performance metrics..."
[Show metrics table/chart]
```

[30 seconds] Conclusion

```
"What I learned... Future improvements... Thank you!"
```

5. Click "Stop Recording"
6. Find video file, upload to YouTube (unlisted) or Google Drive

---

Day 5-7: Update README

Add to README.md:

1. Demo Video Link

```
## 🎥 Demo Video
[📺 Watch Demo](https://youtube.com/...)
```

2. Results

```
## Results

| Metric | Value |
|--------|-------|
| Accuracy | 87.3% |
| Inference Time | 0.05s |
```

3. Example Images

```
## Example Results

![Success](results/success1.jpg)
![Failure](results/failure1.jpg)
```

Week 14 (Nov 27 - Dec 4): Final Polish

Day 1-2: Clean Up Code

Checklist:

- Add comments to your code
- Remove unused code/files
- Test everything runs without errors
- Update requirements.txt
- Add docstrings to functions
- Make sure sample data is included

Quick test:

```
# Clone your repo to a fresh location
git clone [your-repo-url] test_folder
cd test_folder

# Try to run it
pip install -r requirements.txt
jupyter notebook notebooks/04_demo.ipynb

# Does it work? Fix any issues!
```

Day 3-4: Make Presentation Slides

10 slides:

1. Title
2. Problem
3. Solution
4. Technical Approach
5. System Architecture
6. Demo (you'll do live demo here!)
7. Results (metrics)
8. Results (examples)
9. Learnings
10. Future Work

Practice your live demo 3 times!

Day 5-7: Final Testing

Test Day Simulation:

- Open your laptop
- Connect to internet
- Open your notebook
- Run all cells
- Does it work? If yes, you're ready!
- If no, fix and test again

Prepare backup:

- Download your demo video to USB drive
- Save screenshots of your results
- Print your slides (just in case)

Week 15 (Dec 4): PRESENTATION DAY! 🎉

Before Class:

- Laptop is charged
- Demo is tested
- Slides are ready
- Backup video on USB
- Deep breath!

During Presentation:

1. Introduce yourself and project (1 min)
2. Explain problem and solution (2 min)

3. **LIVE DEMO** (5 min) - Most important!
4. Show results (2 min)
5. Q&A

If your demo fails:

- Don't panic!
- Play backup video
- Explain what went wrong
- Answer questions anyway

After Class:

🎉 **Congratulations! You did it!** 🎉

🆘 TROUBLESHOOTING COMMON ISSUES

"I can't install packages!"

```
# Try:
pip install --upgrade pip
pip install [package-name]

# If still fails:
conda install [package-name]

# If still fails:
# Use Google Colab instead!
```

"CUDA out of memory!"

```
# Reduce batch size:
model.train(batch=4)  # or batch=2

# Or use CPU:
model.train(device='cpu')  # Will be slower!
```

"My model won't load!"

```
# Check the path:
import os
print(os.path.exists('path/to/model'))
```

```
# Try absolute path:
model = YOLO('/full/path/to/model.pt')
```

"No labels found!"

```
# Check your data.yaml:
# Make sure paths are correct
# Try absolute paths:
path: /content/data
train: /content/data/train/images
```

"Training is too slow!"

- Use smaller model (yolov8n)
- Reduce image size (imgsz=320)
- Use fewer epochs
- Check if GPU is being used:

```
import torch
print(torch.cuda.is_available())  # Should be True
```

💡 FINAL TIPS

1. **Start simple** - Get something working first, then improve
2. **Save often** - Commit to GitHub regularly
3. **Test frequently** - Don't wait until the end
4. **Ask for help** - Instructor, classmates, AI
5. **Document everything** - Future you will thank you
6. **Have fun!** - You're building real AI!

You can do this! Follow the steps, and you'll succeed! 🚀